



INFECTING THE BOOT TO OWN THE KERNEL: BOOTKITS AND ROOTKITS DEVELOPMENT

Demistifying UEFI Bootkits and Kernel-Mode Rootkits Development

[\[in/vazquez-vazquez-alejandro\]](#)

[\[in/mariasanjose\]](#)

DEF CON 33, Las Vegas 2025



WHO ARE WE



GEEK (**G**entleman **E**xploring **uEFI** and OS **K**ernel)



GEEK (**G**irl **E**xploring **E**xploits and **K**ernel threats)



Sensitive Content

This presentation contains age-restricted materials including malware and explicit hooking techniques. By entering, you affirm that you are at least 18 years of age and you consent to viewing “hacker” stuff.

Let me In
This is real malware

No
I prefer to stay with the
humans in the Villages



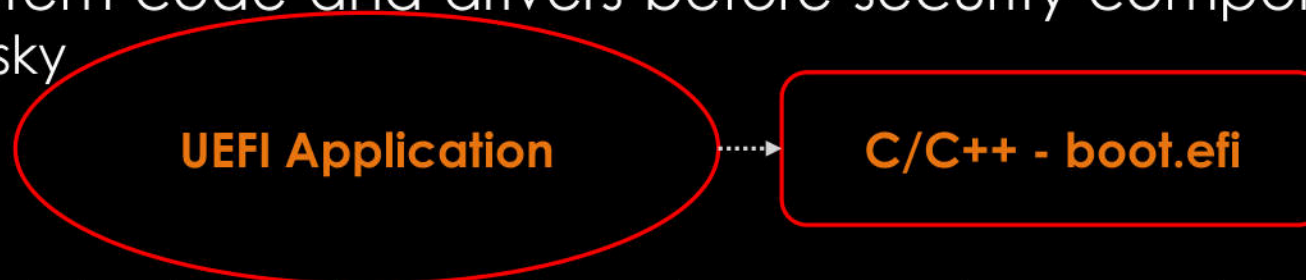
BOOTKITS & ROOTKITS DEMYSTIFIED

- **Bootkit:** Malicious program designed to load as early as possible in the boot process, in order to control all stages of the operating system start up, modifying system code and drivers before security components are loaded.
~ Kaspersky
- **Rootkit:** Sophisticated piece of malware that can add new code to the operating system or delete and edit operating system kernel code.
~ Crowdstrike

BOOTKITS & ROOTKITS DEMYSTIFIED

- **Bootkit**: Malicious program designed to load as early as possible in the boot process, in order to control all stages of the operating system start up, modifying system code and drivers before security components are loaded.

~ Kaspersky

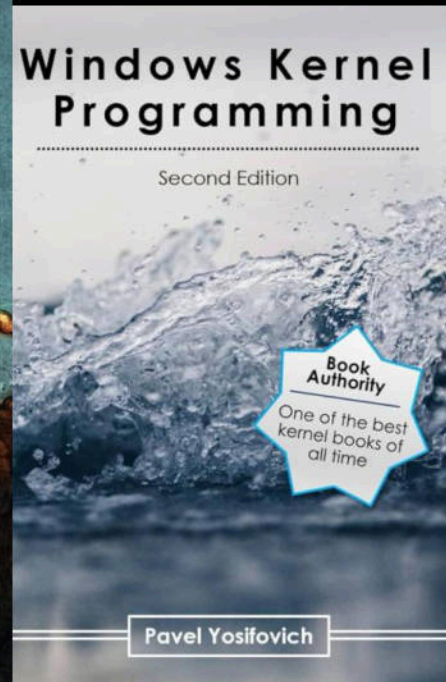
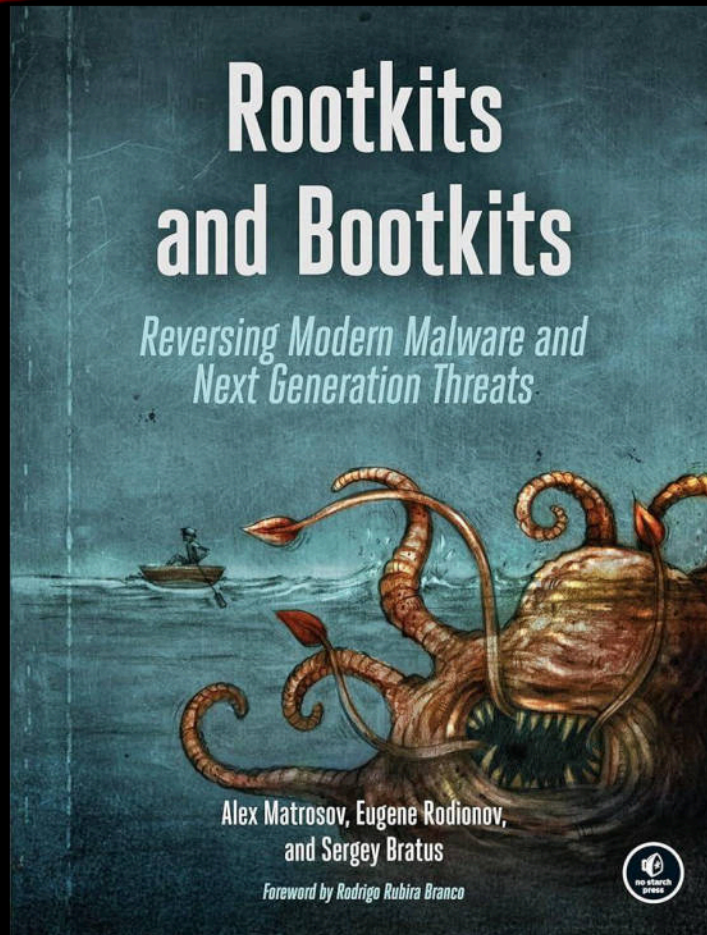


- **Rootkit**: Sophisticated piece of malware that can add new code to the operating system or delete and edit operating system code.

~ CrowdStrike



BOOTKITS & ROOTKITS DEMYSTIFIED



to load as early as possible in the boot system start up.

Windows Internals

Part 1

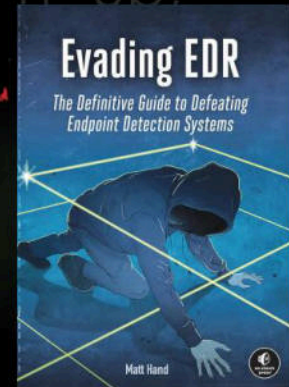
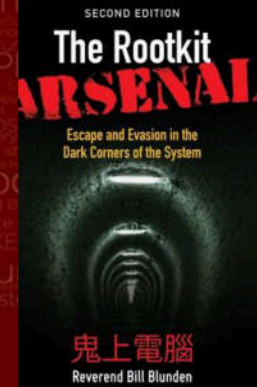
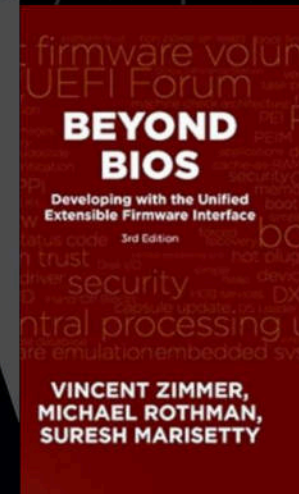
System architecture, processes, threads, memory management, and more

Professional



Pavel Yosifovich
Alex Ionescu
Mark E. Russinovich
David A. Solomon

Microsoft
7
SEVENTH EDITION



Kernel-Mode Driver

C/C++ - driver.sys

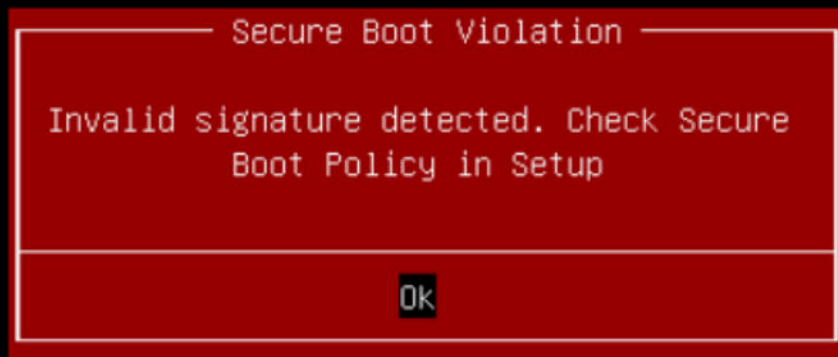


SECURITY MECHANISMS TO BYPASS

- * [*Anti-Bootkit Installation*] - Prevent installation of malicious UEFI applications
- * [*Anti-Rootkit Installation*] - Prevent installation of malicious Kernel-mode drivers

SECURITY MECHANISMS TO BYPASS

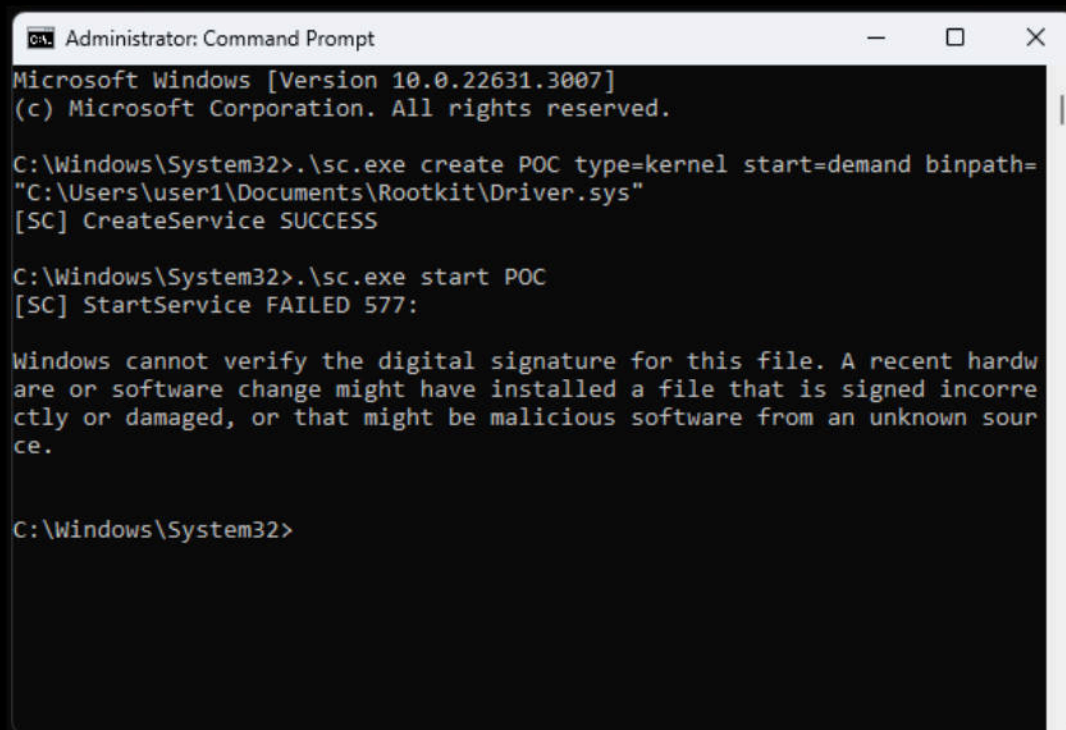
- * [*Anti-Bootkit Installation*] - Prevent installation of malicious UEFI applications
- SecureBoot
 - Only software trusted by the Original Manufacturer
 - Firmware checks the signature of UEFI firmware drivers, EFI applications and SO



CVE-2025-33043	(SMRAM write)
CVE-2024-8105	(PKfail)
CVE-2024-7344	(Custom PE loader)
CVE-2023-40238	(LogoFAIL Image parsing)
CVE-2022-21894	(Baton Drop)

SECURITY MECHANISMS TO BYPASS

- * *[Anti-Rootkit Installation]* - Prevent installation of malicious Kernel-mode drivers
- Driver Signature Enforcement (DSE)
Windows won't run drivers not certified by Microsoft



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.22631.3007]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>.sc.exe create POC type=kernel start=demand binpath=
"C:\Users\user1\Documents\Rootkit\Driver.sys"
[SC] CreateService SUCCESS

C:\Windows\System32>.sc.exe start POC
[SC] StartService FAILED 577:

Windows cannot verify the digital signature for this file. A recent hardw
are or software change might have installed a file that is signed incorre
ctly or damaged, or that might be malicious software from an unknown sour
ce.

C:\Windows\System32>
```

Bring Your Own Vulnerable Driver (BYOVD)

RTCore64.sys

Viragt64.sys

dbutil_2_3.sys

loldrivers

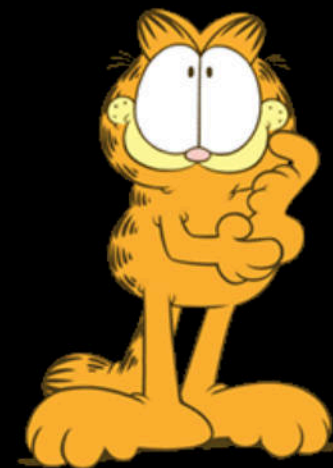
CHECKPOINT

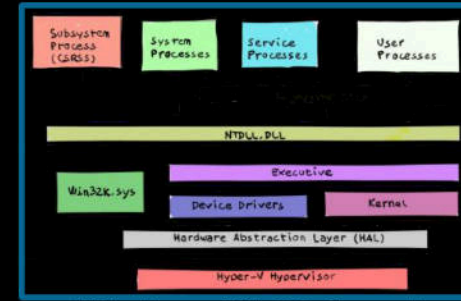
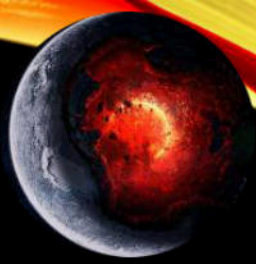
Bootkit/Rootkit Development

- Bootkit = Malicious UEFI Application
- Rootkit = Malicious Kernel Mode Driver
- Protection Mechanisms
 - SecureBoot
 - DSE
 - PatchGuard
 - Vulnerable driver blocklist

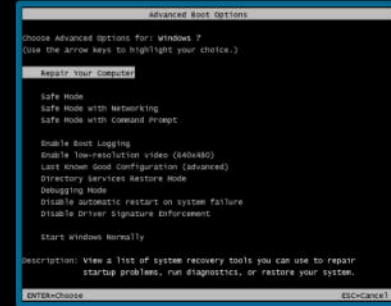
.....

LET ME THINK...

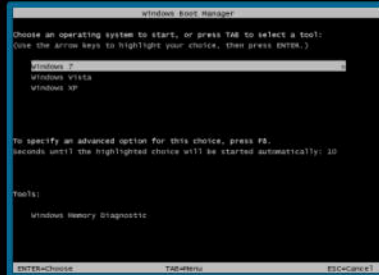




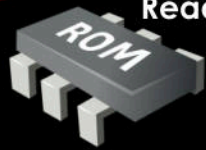
Windows NT OS Kernel
%SystemRoot%\system32\
ntoskrnl.exe



Windows OS Loader
%SystemRoot%\system32\
winload.efi



Windows Boot Manager
\\EFI\\Microsoft\\Boot\\
bootmgfw.efi



Read Instructions



Power ON

POST



UEFI Firmware



Boot Information



EFI System Partition



Boot order

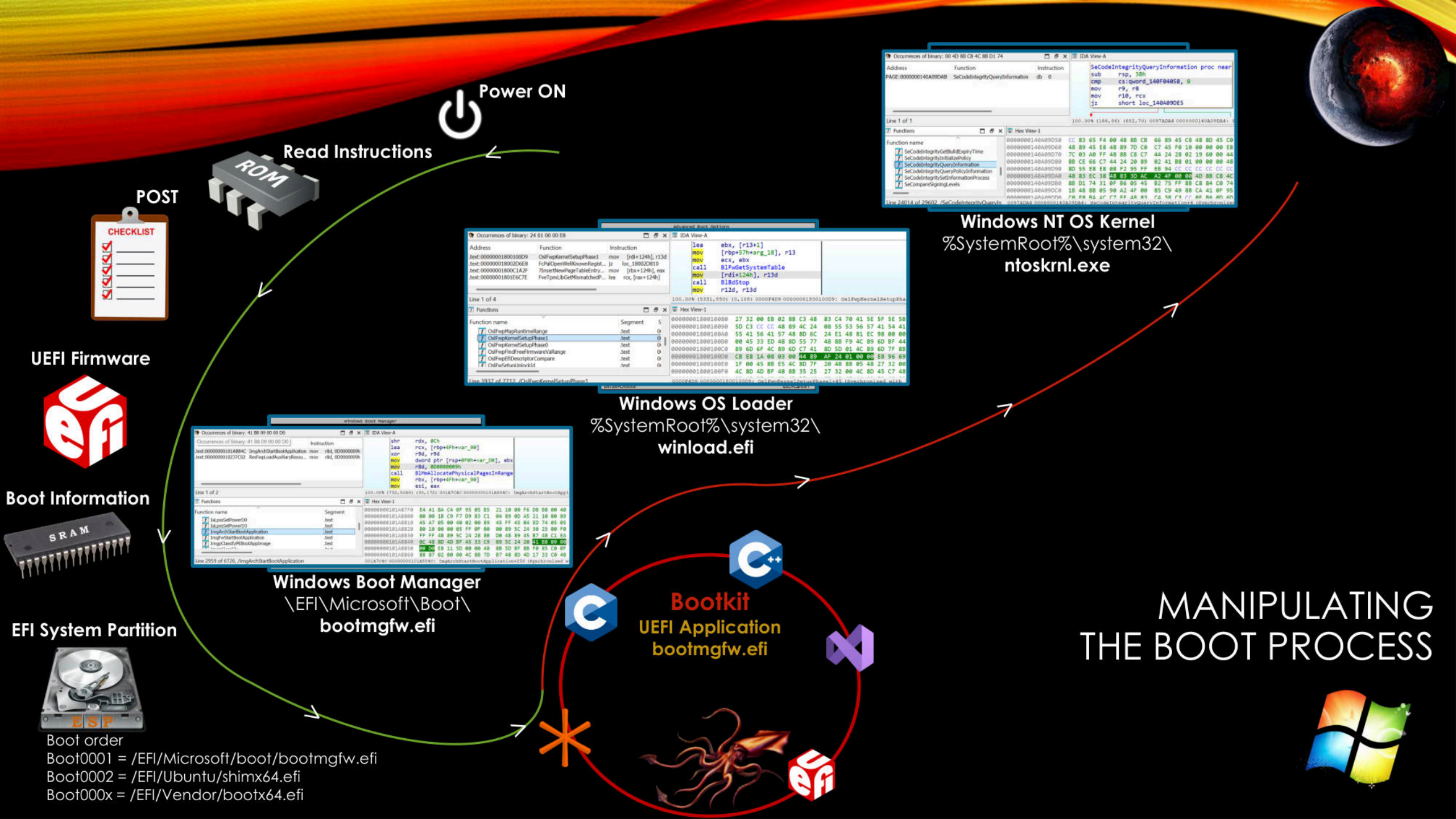
Boot0001 = /EFI/Microsoft/boot/bootmgfw.efi

Boot0002 = /EFI/Ubuntu/shimx64.efi

Boot000x = /EFI/Vendor/bootx64.efi

UNDERSTANDING
THE BOOT PROCESS





Power ON

Read Instructions

POST

UEFI Firmware

Boot Information

EFI System Partition

Windows Boot Manager
\\EFI\\Microsoft\\Boot\\
bootmgfw.efi

Windows OS Loader
%SystemRoot%\\system32\\
winload.efi

Windows NT OS Kernel
%SystemRoot%\\system32\\
ntoskrnl.exe

MANIPULATING
THE BOOT PROCESS

Boot order
Boot0001 = /EFI/Microsoft/boot/bootmgfw.efi
Boot0002 = /EFI/Ubuntu/shimx64.efi
Boot000x = /EFI/Vendor/bootx64.efi

```
// Find function pattern
Status = FindPattern(TargetFunctionSignature, 0xCC, sizeof(TargetFunctionSignature), (UINT8*)ImageBase...
```

```
// Find function start
OriginalFunctionPointer = (VOID*)FindStartAddress(ImageBase, NtHeaders, Found);
```

```
// Hook bootmgfw.efi!Function, winload.efi!Function
VOID* HookAddress = (VOID*)&MyHookHandlerFunction;
```

```
// Backup original function bytes
CopyMem(BackupOriginalBytes, OriginalFunctionPointer, sizeof(HookTemplate));
```

```
// Place hook trampoline at the start of the original function
CopyMem(OriginalFunctionPointer, HookTemplate, sizeof(HookTemplate));
```

```
// Write hook handler address into trampoline template
CopyMem((UINT8*) OriginalFunctionPointer + HookTemplateAddressOffset, (UINTN*)&HookAddress, sizeof(UINTN));
```

Boot Information



EFI System Partition



Boot order
Boot0001 = /EFI/Microsoft/boot/bootmgfw.efi
Boot0002 = /EFI/Ubuntu/shimx64.efi
Boot000x = /EFI/Vendor/bootx64.efi

Windows Boot Manager
\\EFI\\Microsoft\\Boot\\
bootmgfw.efi

Windows OS Loader

%SystemRoot%\system32\\
winload.efi

Bootkit
UEFI Application
bootmgfw.efi

Windows NT OS Kernel
%SystemRoot%\system32\\
ntoskrnl.exe

MANIPULATING
THE BOOT PROCESS





Power ON

Read Instructions

POST

UEFI Firmware

Boot Information

EFI System Partition

boot → bootmgfw.efi

- ExitBootServices();
- gBS->LoadImage();

bootmgfw.efi → winload.efi

- ImgArchStartBootApplication();
- Archpx64TransferTo64BitApplicationAsm();

winload.efi → ntoskrnl.exe

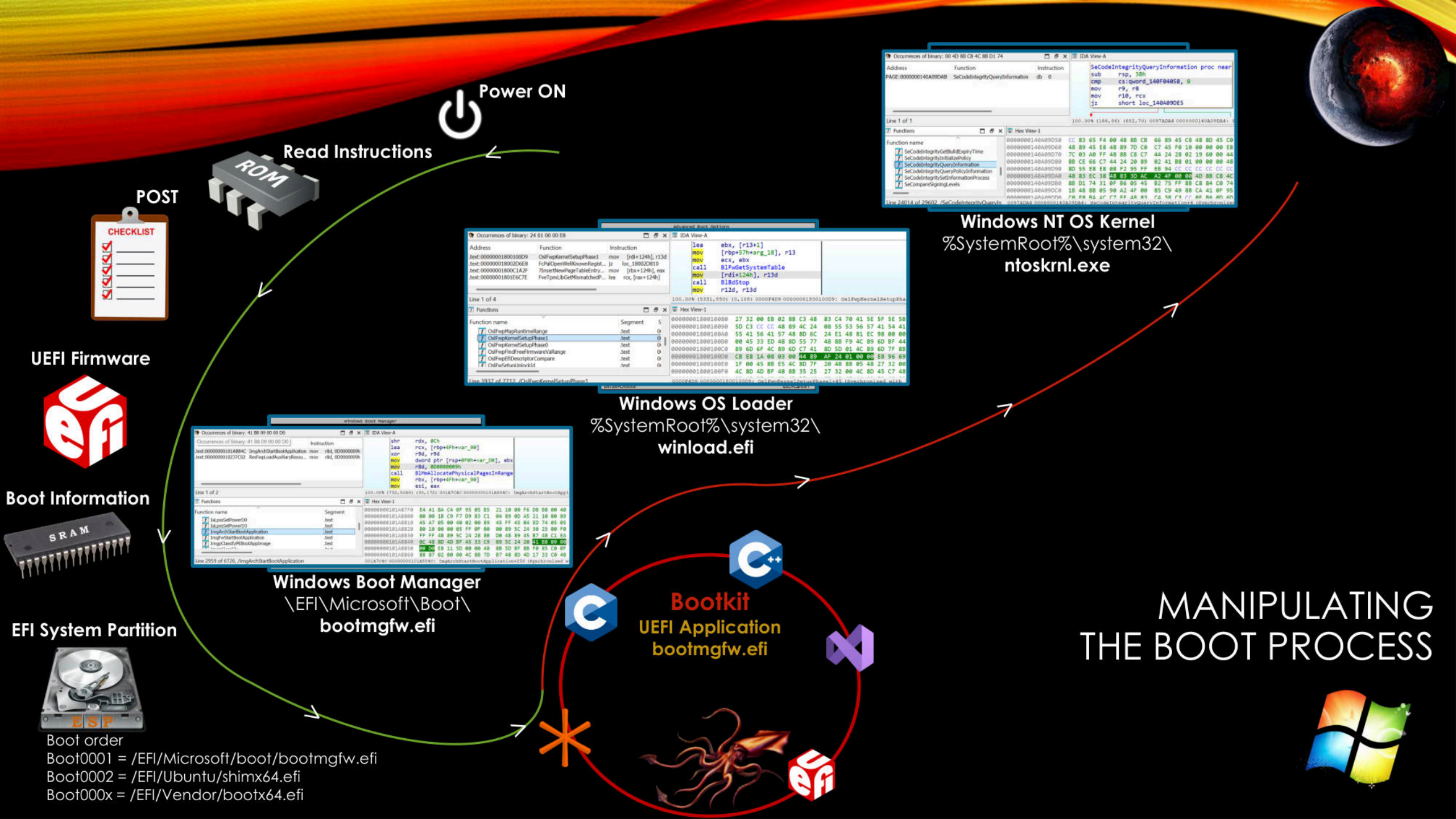
- OslArchTransferToKernel();
- OslFwpKernelSetupPhase1();
- BllmAllocateImageBuffer();

ntoskrnl.exe → user

- CmGetSystemDriverList();
- SeCodeIntegrityQueryInformation();

MANIPULATING THE BOOT PROCESS





Power ON

Read Instructions

POST

UEFI Firmware

Boot Information

EFI System Partition

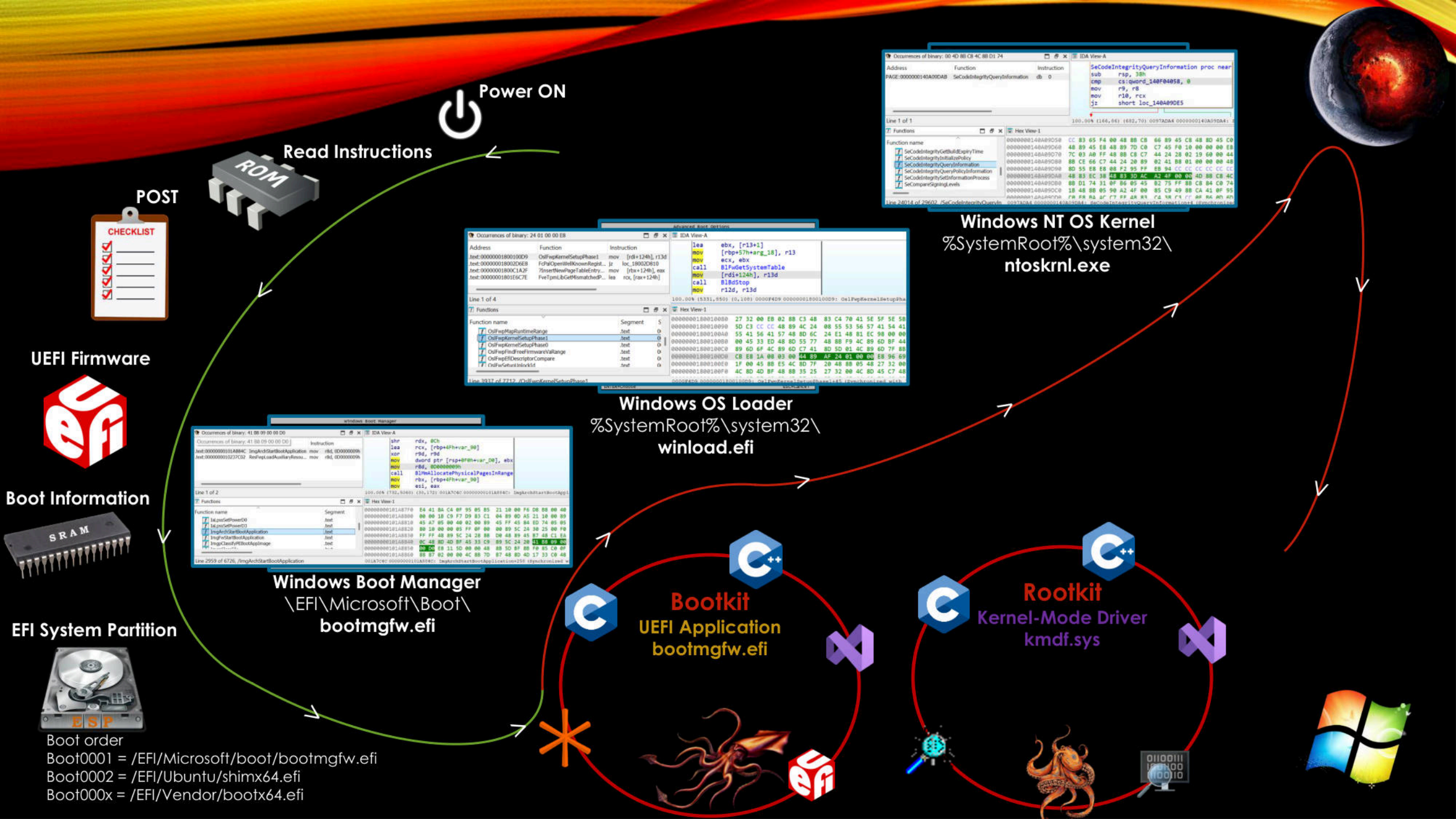
Windows Boot Manager
\\EFI\\Microsoft\\Boot\\
bootmgfw.efi

Windows OS Loader
%SystemRoot%\\system32\\
winload.efi

Windows NT OS Kernel
%SystemRoot%\\system32\\
ntoskrnl.exe

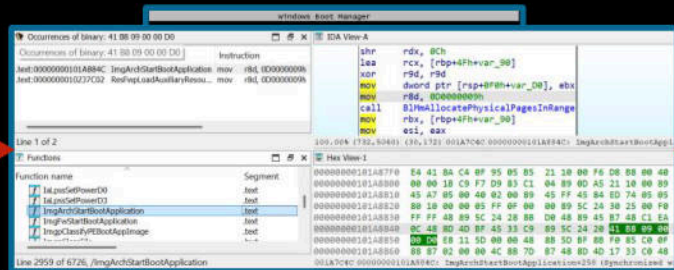
MANIPULATING
THE BOOT PROCESS

Boot order
Boot0001 = /EFI/Microsoft/boot/bootmgfw.efi
Boot0002 = /EFI/Ubuntu/shimx64.efi
Boot000x = /EFI/Vendor/bootx64.efi

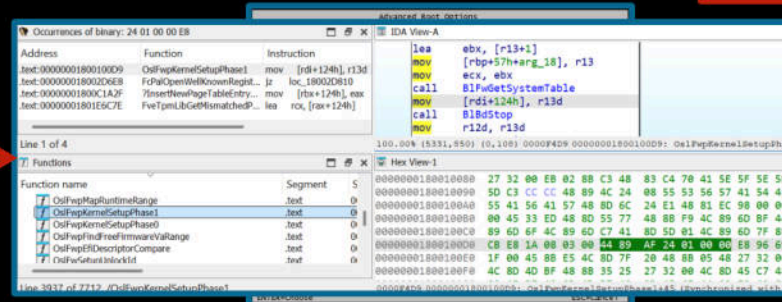


DEMO

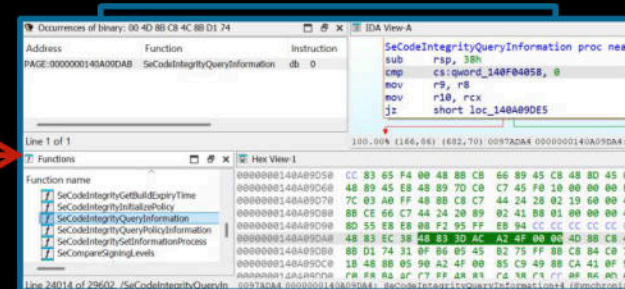




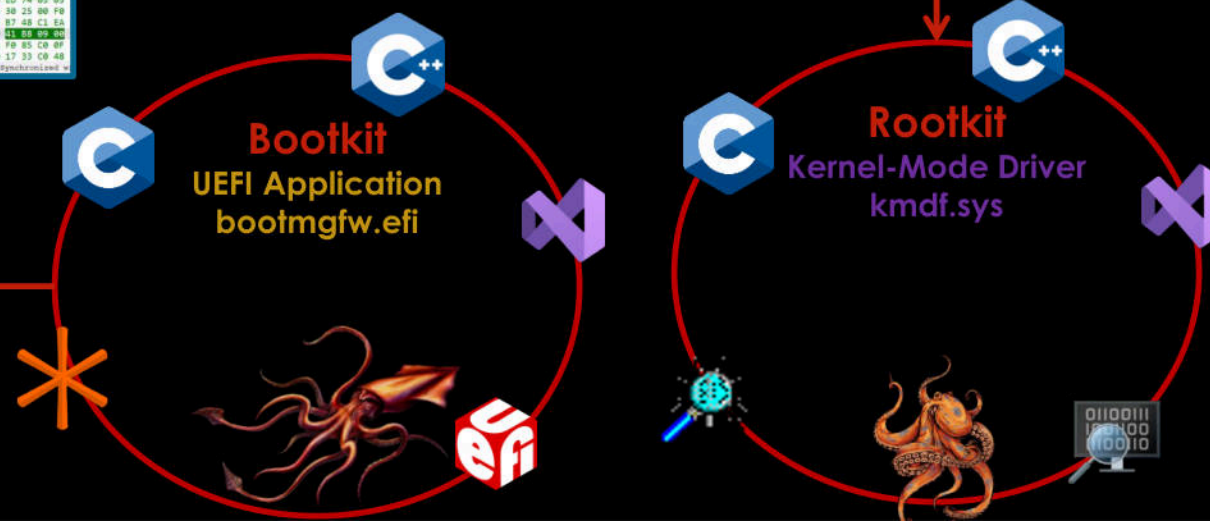
Windows Boot Manager
\\EFI\\Microsoft\\Boot\\
bootmgfw.efi

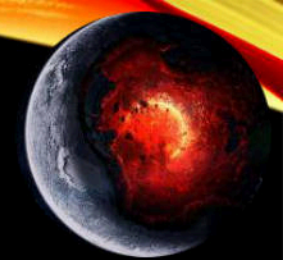


Windows OS Loader
%SystemRoot%\system32\\
winload.efi



Windows NT OS Kernel
%SystemRoot%\system32\\
ntoskrnl.exe

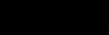




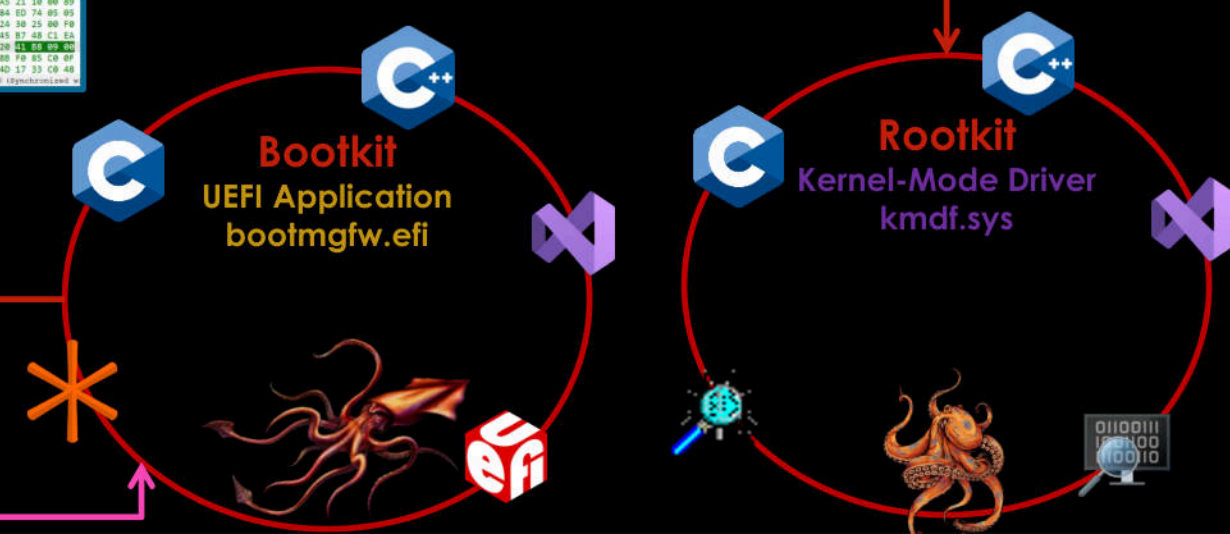
The screenshot displays the Immunity Debugger interface with three main panes:

- Occurrences of binary: 24 01 00 00 EB**: Shows assembly instructions for the function `__OsiPwKernSetupPhase1`. The instructions are:
 - `mov edi, [edx+124h], r13d`
 - `loc.19002D810: rcpOpnGetWellKnownRegist... jrcxz [ebx+124h], eax`
 - `mov rax, [edx+124h]`
 - `lea r13d, r13d`
- Hex View-1**: Shows the corresponding hex dump of the assembly code, with addresses ranging from `00000000180010000` to `0000000018001000F`.
- Function List**: A list of functions in the module, including `__OsiPwKernSetupPhase1`, `__OsiPwKernSetupPhase2`, `__OsiPwKernSetupPhase3`, `__OsiPwKernSetupPhase4`, `__OsiPwKernSetupPhase5`, `__OsiPwKernSetupPhase6`, `__OsiPwKernSetupPhase7`, `__OsiPwKernSetupPhase8`, `__OsiPwKernSetupPhase9`, `__OsiPwKernSetupPhase10`, `__OsiPwKernSetupPhase11`, `__OsiPwKernSetupPhase12`, `__OsiPwKernSetupPhase13`, `__OsiPwKernSetupPhase14`, `__OsiPwKernSetupPhase15`, `__OsiPwKernSetupPhase16`, `__OsiPwKernSetupPhase17`, `__OsiPwKernSetupPhase18`, `__OsiPwKernSetupPhase19`, `__OsiPwKernSetupPhase20`, `__OsiPwKernSetupPhase21`, `__OsiPwKernSetupPhase22`, `__OsiPwKernSetupPhase23`, `__OsiPwKernSetupPhase24`, `__OsiPwKernSetupPhase25`, `__OsiPwKernSetupPhase26`, `__OsiPwKernSetupPhase27`, `__OsiPwKernSetupPhase28`, `__OsiPwKernSetupPhase29`, `__OsiPwKernSetupPhase30`, `__OsiPwKernSetupPhase31`, `__OsiPwKernSetupPhase32`, `__OsiPwKernSetupPhase33`, `__OsiPwKernSetupPhase34`, `__OsiPwKernSetupPhase35`, `__OsiPwKernSetupPhase36`, `__OsiPwKernSetupPhase37`, `__OsiPwKernSetupPhase38`, `__OsiPwKernSetupPhase39`, `__OsiPwKernSetupPhase40`, `__OsiPwKernSetupPhase41`, `__OsiPwKernSetupPhase42`, `__OsiPwKernSetupPhase43`, `__OsiPwKernSetupPhase44`, `__OsiPwKernSetupPhase45`, `__OsiPwKernSetupPhase46`, `__OsiPwKernSetupPhase47`, `__OsiPwKernSetupPhase48`, `__OsiPwKernSetupPhase49`, `__OsiPwKernSetupPhase50`, `__OsiPwKernSetupPhase51`, `__OsiPwKernSetupPhase52`, `__OsiPwKernSetupPhase53`, `__OsiPwKernSetupPhase54`, `__OsiPwKernSetupPhase55`, `__OsiPwKernSetupPhase56`, `__OsiPwKernSetupPhase57`, `__OsiPwKernSetupPhase58`, `__OsiPwKernSetupPhase59`, `__OsiPwKernSetupPhase60`, `__OsiPwKernSetupPhase61`, `__OsiPwKernSetupPhase62`, `__OsiPwKernSetupPhase63`, `__OsiPwKernSetupPhase64`, `__OsiPwKernSetupPhase65`, `__OsiPwKernSetupPhase66`, `__OsiPwKernSetupPhase67`, `__OsiPwKernSetupPhase68`, `__OsiPwKernSetupPhase69`, `__OsiPwKernSetupPhase70`, `__OsiPwKernSetupPhase71`, `__OsiPwKernSetupPhase72`, `__OsiPwKernSetupPhase73`, `__OsiPwKernSetupPhase74`, `__OsiPwKernSetupPhase75`, `__OsiPwKernSetupPhase76`, `__OsiPwKernSetupPhase77`, `__OsiPwKernSetupPhase78`, `__OsiPwKernSetupPhase79`, `__OsiPwKernSetupPhase80`, `__OsiPwKernSetupPhase81`, `__OsiPwKernSetupPhase82`, `__OsiPwKernSetupPhase83`, `__OsiPwKernSetupPhase84`, `__OsiPwKernSetupPhase85`, `__OsiPwKernSetupPhase86`, `__OsiPwKernSetupPhase87`, `__OsiPwKernSetupPhase88`, `__OsiPwKernSetupPhase89`, `__OsiPwKernSetupPhase90`, `__OsiPwKernSetupPhase91`, `__OsiPwKernSetupPhase92`, `__OsiPwKernSetupPhase93`, `__OsiPwKernSetupPhase94`, `__OsiPwKernSetupPhase95`, `__OsiPwKernSetupPhase96`, `__OsiPwKernSetupPhase97`, `__OsiPwKernSetupPhase98`, `__OsiPwKernSetupPhase99`, `__OsiPwKernSetupPhase100`, `__OsiPwKernSetupPhase101`, `__OsiPwKernSetupPhase102`, `__OsiPwKernSetupPhase103`, `__OsiPwKernSetupPhase104`, `__OsiPwKernSetupPhase105`, `__OsiPwKernSetupPhase106`, `__OsiPwKernSetupPhase107`, `__OsiPwKernSetupPhase108`, `__OsiPwKernSetupPhase109`, `__OsiPwKernSetupPhase110`, `__OsiPwKernSetupPhase111`, `__OsiPwKernSetupPhase112`, `__OsiPwKernSetupPhase113`, `__OsiPwKernSetupPhase114`, `__OsiPwKernSetupPhase115`, `__OsiPwKernSetupPhase116`, `__OsiPwKernSetupPhase117`, `__OsiPwKernSetupPhase118`, `__OsiPwKernSetupPhase119`, `__OsiPwKernSetupPhase120`, `__OsiPwKernSetupPhase121`, `__OsiPwKernSetupPhase122`, `__OsiPwKernSetupPhase123`, `__OsiPwKernSetupPhase124`, `__OsiPwKernSetupPhase125`, `__OsiPwKernSetupPhase126`, `__OsiPwKernSetupPhase127`, `__OsiPwKernSetupPhase128`, `__OsiPwKernSetupPhase129`, `__OsiPwKernSetupPhase130`, `__OsiPwKernSetupPhase131`, `__OsiPwKernSetupPhase132`, `__OsiPwKernSetupPhase133`, `__OsiPwKernSetupPhase134`, `__OsiPwKernSetupPhase135`, `__OsiPwKernSetupPhase136`, `__OsiPwKernSetupPhase137`, `__OsiPwKernSetupPhase138`, `__OsiPwKernSetupPhase139`, `__OsiPwKernSetupPhase140`, `__OsiPwKernSetupPhase141`, `__OsiPwKernSetupPhase142`, `__OsiPwKernSetupPhase143`, `__OsiPwKernSetupPhase144`, `__OsiPwKernSetupPhase145`, `__OsiPwKernSetupPhase146`, `__OsiPwKernSetupPhase147`, `__OsiPwKernSetupPhase148`, `__OsiPwKernSetupPhase149`, `__OsiPwKernSetupPhase150`, `__OsiPwKernSetupPhase151`, `__OsiPwKernSetupPhase152`, `__OsiPwKernSetupPhase153`, `__OsiPwKernSetupPhase154`, `__OsiPwKernSetupPhase155`, `__OsiPwKernSetupPhase156`, `__OsiPwKernSetupPhase157`, `__OsiPwKernSetupPhase158`, `__OsiPwKernSetupPhase159`, `__OsiPwKernSetupPhase160`, `__OsiPwKernSetupPhase161`, `__OsiPwKernSetupPhase162`, `__OsiPwKernSetupPhase163`, `__OsiPwKernSetupPhase164`, `__OsiPwKernSetupPhase165`, `__OsiPwKernSetupPhase166`, `__OsiPwKernSetupPhase167`, `__OsiPwKernSetupPhase168`, `__OsiPwKernSetupPhase169`, `__OsiPwKernSetupPhase170`, `__OsiPwKernSetupPhase171`, `__OsiPwKernSetupPhase172`, `__OsiPwKernSetupPhase173`, `__OsiPwKernSetupPhase174`, `__OsiPwKernSetupPhase175`, `__OsiPwKernSetupPhase176`, `__OsiPwKernSetupPhase177`, `__OsiPwKernSetupPhase178`, `__OsiPwKernSetupPhase179`, `__OsiPwKernSetupPhase180`, `__OsiPwKernSetupPhase181`, `__OsiPwKernSetupPhase182`, `__OsiPwKernSetupPhase183`, `__OsiPwKernSetupPhase184`, `__OsiPwKernSetupPhase185`, `__OsiPwKernSetupPhase186`, `__OsiPwKernSetupPhase187`, `__OsiPwKernSetupPhase188`, `__OsiPwKernSetupPhase189`, `__OsiPwKernSetupPhase190`, `__OsiPwKernSetupPhase191`, `__OsiPwKernSetupPhase192`, `__OsiPwKernSetupPhase193`, `__OsiPwKernSetupPhase194`, `__OsiPwKernSetupPhase195`, `__OsiPwKernSetupPhase196`, `__OsiPwKernSetupPhase197`, `__OsiPwKernSetupPhase198`, `__OsiPwKernSetupPhase199`, `__OsiPwKernSetupPhase200`, `__OsiPwKernSetupPhase201`, `__OsiPwKernSetupPhase202`, `__OsiPwKernSetupPhase203`, `__OsiPwKernSetupPhase204`, `__OsiPwKernSetupPhase205`, `__OsiPwKernSetupPhase206`, `__OsiPwKernSetupPhase207`, `__OsiPwKernSetupPhase208`, `__OsiPwKernSetupPhase209`, `__OsiPwKernSetupPhase210`, `__OsiPwKernSetupPhase211`, `__OsiPwKernSetupPhase212`, `__OsiPwKernSetupPhase213`, `__OsiPwKernSetupPhase214`, `__OsiPwKernSetupPhase215`, `__OsiPwKernSetupPhase216`, `__OsiP`

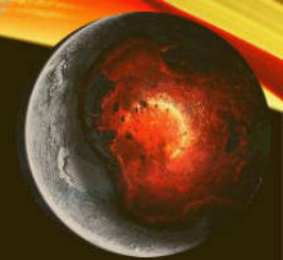
Windows OS Loader
%SystemRoot%\system32\
winload.efi

[illegible]

Bootkit
UEFI Application
bootmgfw.efi



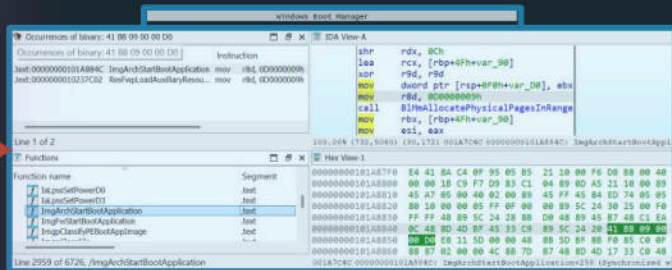
- UEFI Specification 2.11
 - Introduction
 - Overview
 - Boot Manager
 - BIOS System Table
 - GUID Partition Table (GPT) Disk Layout
 - Block Translation Table (BTT) Layout
 - Services – Boot Services
 - Services – Runtime Services
 - Protocols – EFI Loaded Image
-
- Network Protocols – SNP, PXE, BIS
- HTTP Boot
- Network Protocols - Managed Network
- Network Protocols – Bluetooth
- Network Protocols – VLAN, EAP, and Supplicant
- Network Protocols – TCP, IP, IPsec, TLS and Configurations
- Network Protocols – ARP, DHCP, HTTP and REST
- Network Protocols – UDP and TCP



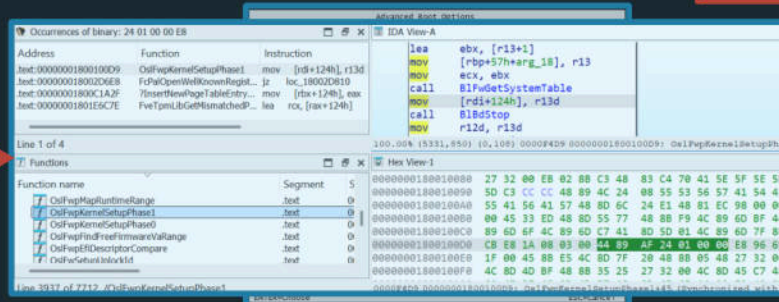
UEFI Specification

2.11

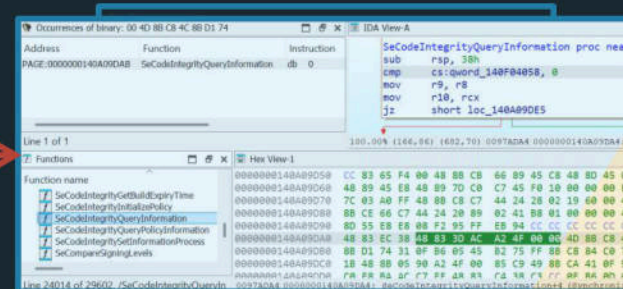
1. Introduction
2. Overview
3. Boot Manager
4. EFI System Table
5. GUID Partition Table (GPT) Disk Layout
6. Block Translation Table (BTT) Layout
7. Services – Boot Services
8. Services – Runtime Services
9. Protocols - EFI Loaded Image
24. Network Protocols – SNP, PXE, BIS and HTTP Boot
25. Network Protocols - Managed Network
26. Network Protocols – Bluetooth
27. Network Protocols – VLAN, EAP, Wi-Fi and Supplicant
28. Network Protocols – TCP, IP, IPsec, FTP, TLS and Configurations
29. Network Protocols – ARP, DHCP, DNS, HTTP and REST
30. Network Protocols – UDP and MTFTP



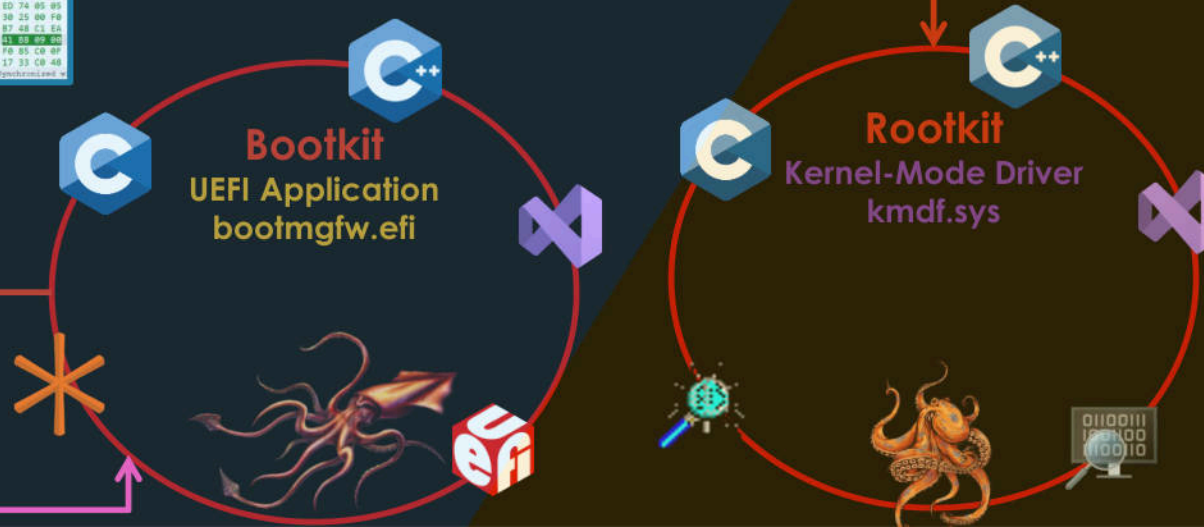
Windows Boot Manager
\\EFI\\Microsoft\\Boot\\
bootmgfw.efi

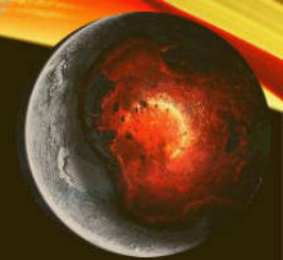


Windows OS Loader
%SystemRoot%\system32\\
winload.efi



Windows NT OS Kernel
%SystemRoot%\system32\\
ntoskrnl.exe

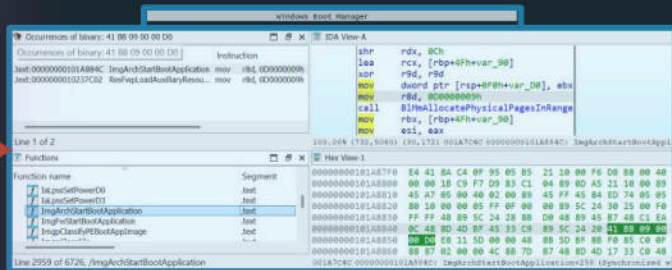




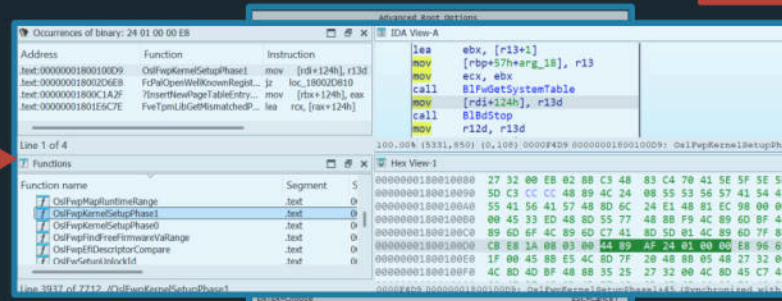
UEFI Specification

2.11

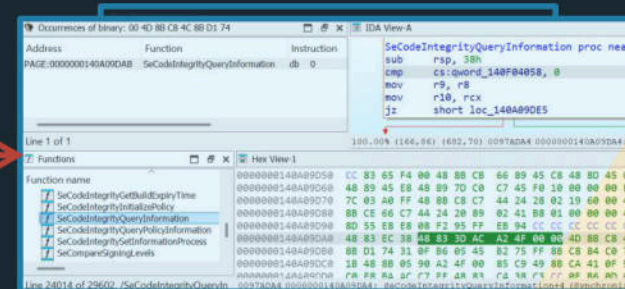
1. Introduction
2. Overview
3. Boot Manager
4. EFI System Table
5. GUID Partition Table (GPT) Disk Layout
6. Block Translation Table (BTT) Layout
7. Services – Boot Services
8. Services – Runtime Services
9. Protocols - EFI Loaded Image
24. Network Protocols – SNP, PXE, BIS and HTTP Boot
25. Network Protocols - Managed Network
26. Network Protocols – Bluetooth
27. Network Protocols – VLAN, EAP, Wi-Fi and Supplicant
28. Network Protocols – TCP, IP, IPsec, FTP, TLS and Configurations
29. Network Protocols – ARP, DHCP, DNS, HTTP and REST
30. Network Protocols – UDP and MTFTP



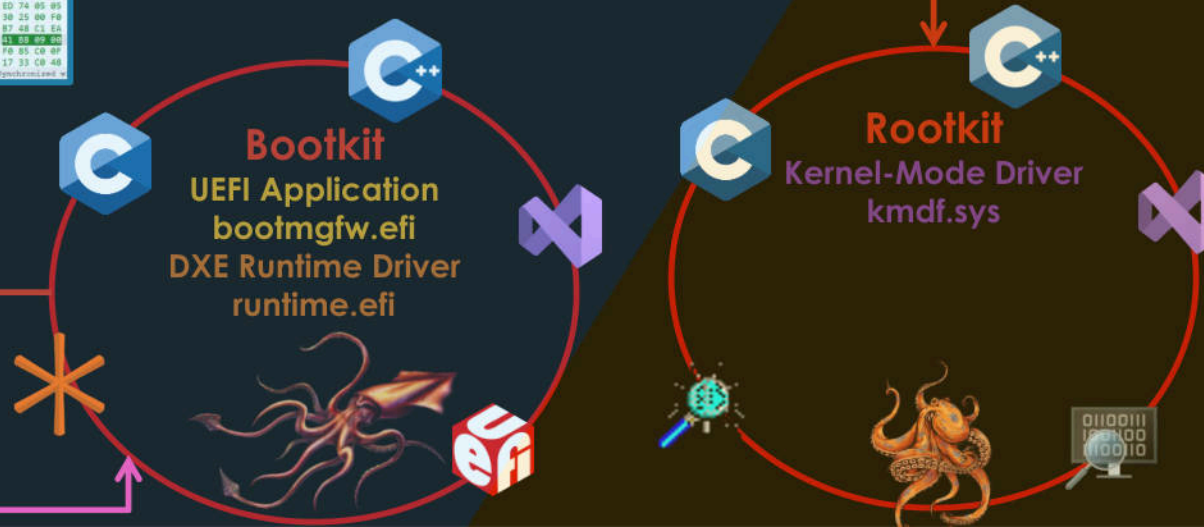
Windows Boot Manager
\\EFI\\Microsoft\\Boot\\
bootmgfw.efi

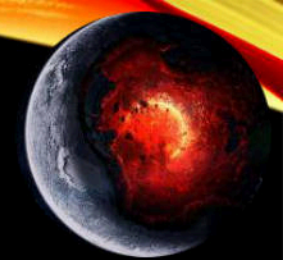


Windows OS Loader
%SystemRoot%\system32\\
winload.efi



Windows NT OS Kernel
%SystemRoot%\system32\\
ntoskrnl.exe



[illegible][illegible]

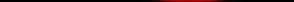
The screenshot displays the Immunity Debugger interface with two main panes. The top pane shows assembly code for a function named `!ImgrGetPowerOn`. The code includes instructions like `mov rdi, 0000000000000000`, `xor r9d, r9d`, `shrd ptr, [rsi+40H+var_00], ebx`, `call !BjMmLocatePhysicalPagesInRange`, and `esi, eax`. The bottom pane shows a hex dump of memory starting at address `00401000`, with values ranging from `54 41 86 C4 0F 95 05 01` to `04 0A 8D 42 17 33 08 00`.

C++

Bootkit

**UEFI Application
bootmgfw.efi**

**DXE Runtime Driver
runtime.efi**



1. Introduction
2. Overview
3. Boot Manager
4. EFI System Table
5. GUID Partition Table (GPT) Disk Layout
6. Block Translation Table (BTT) Layout
7. Services – Boot Services
8. Services – Runtime Services
9. Protocols - EFI Loaded Image
-
24. Network Protocols – SNP, PXE, BIS and HTTP Boot
25. Network Protocols - Managed Network
26. Network Protocols – Bluetooth
27. Network Protocols – VLAN, EAP, Wi-Fi and Supplicant
28. Network Protocols – TCP, IP, IPsec, FTP, TLS and Configurations
29. Network Protocols – ARP, DHCP, DNS, HTTP and REST
30. Network Protocols – UDP and MTFTP

DEMO





- UEFI Specification 2.11
- 1. Introduction
- 2. Overview
- 3. Boot Manager
- 4. EFI System Table
- 5. GUID Partition Table (GPT) Disk Layout
- 6. Block Translation Table (BTT) Layout
- 7. Services – Boot Services
- 8. Services – Runtime Services
- 9. Protocols - EFI Loaded Image
- 24. Network Protocols – SNP, PXE, BIS and HTTP Boot
- 25. Network Protocols - Managed Network
- 26. Network Protocols – Bluetooth
- 27. Network Protocols – VLAN, EAP, Wi-Fi and Supplicant
- 28. Network Protocols – TCP, IP, IPsec, FTP, TLS and Configurations
- 29. Network Protocols – ARP, DHCP, DNS, HTTP and REST
- 30. Network Protocols – UDP and MTFTP

ImgArchStartBootApplication();
Archpx64TransferTo64BitApplicationAsm();

Line 259 of 6726: /imgArchStartBootApplication

Windows Boot Manager
\\EFI\\Microsoft\\Boot\\
bootmgfw.efi

ExitBootServices();
gBS->LoadImage();

OslArchTransferToKernel();
OslFwpKernelSetupPhase1();
BllmgAllocatelmageBuffer();

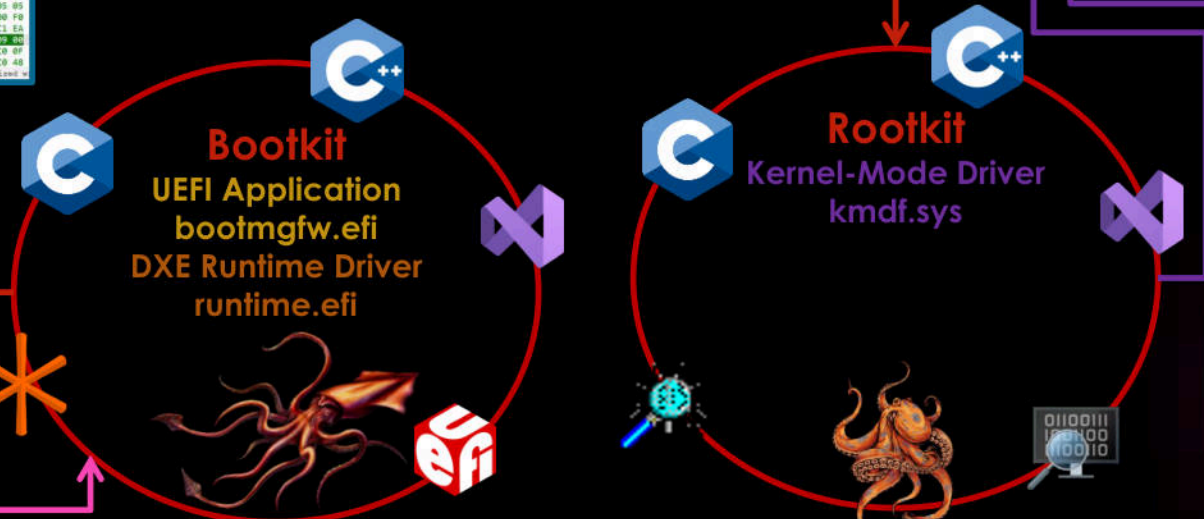
Line 3017 of 7712: /OslFwpKernelSetupPhase1

Windows OS Loader
%SystemRoot%\system32\\
winload.efi

CmGetSystemDriverList();
SeCodeIntegrityQueryInformation();

Line 1 of 1

Windows NT OS Kernel
%SystemRoot%\system32\\
ntoskrnl.exe





- UEFI Specification
2.11
1. Introduction
 2. Overview
 3. Boot Manager
 4. EFI System Table
 5. GUID Partition Table (GPT) Disk Layout
 6. Block Translation Table (BTT) Layout
 7. Services – Boot Services
 8. Services – Runtime Services
 9. Protocols - EFI Loaded Image
 24. Network Protocols – SNP, PXE, BIS and HTTP Boot
 25. Network Protocols - Managed Network
 26. Network Protocols – Bluetooth
 27. Network Protocols – VLAN, EAP, Wi-Fi and Supplicant
 28. Network Protocols – TCP, IP, IPsec, FTP, TLS and Configurations
 29. Network Protocols – ARP, DHCP, DNS, HTTP and REST
 30. Network Protocols – UDP and MTFTP

ImgArchStartBootApplication();
Archpx64TransferTo64BitApplicationAsm();

Windows Boot Manager
\\EFI\\Microsoft\\Boot\\bootmgfw.efi

Occurrences of binary: 41 8B 09 00 00 00
Function name: InitPowerOn
Instruction: shr rdx, 8Ch
lea rcx, [rbp+47h+var_00]
xor rdx, rdx
mov dword ptr [rsp+8F0h+var_00], ebx
mov rdx, 80000000h
call BfWInitPhysicalPagesInRange
mov rdx, [rbp+47h+var_00]
mov esi, eax

Windows Boot Manager
\\EFI\\Microsoft\\Boot\\bootmgfw.efi

ExitBootServices();
gBS->LoadImage();

OslArchTransferToKernel();
OslFwpKernelSetupPhase1();
BllmgAllocateImageBuffer();

Windows OS Loader
%SystemRoot%\system32\winload.efi

Occurrences of binary: 24 01 00 00 E8
Function name: OslFwpKernelSetupPhase1
Instruction: lea ebx, [r13+1]
mov [rbp+57h+arg_18], r13
mov ecx, ebx
call BllFwGetSystemTable
mov [rdi+124h], r13d
call BllStop
mov r12d, r13d

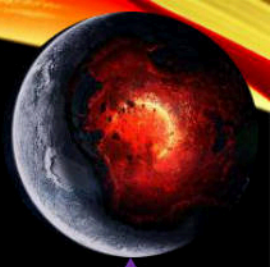
Bootkit
UEFI Application
bootmgfw.efi
DXE Runtime Driver
runtime.efi

CmGetSystemDriverList();
SeCodeIntegrityQueryInformation();

Windows NT OS Kernel
%SystemRoot%\system32\ntoskrnl.exe

Occurrences of binary: 00 4D 8B C8 4C 8B D1 74
Function: SeCodeIntegrityQueryInformation
Instruction: sub rsp, 38h
cmp cs:Quad_140F0405B, 0
mov r9, r8
mov r10, rcx
jz short loc_140A89D05

Rootkit
Kernel-Mode Driver
kmdf.sys
Filter Driver
wfp_wsk_minifilter.sys
Console App
loctl_irp.exe



Windows OS Loader
%SystemRoot%\system32\winload.efi

Occurrences of binary: 41 8B 09 00 00 00 00
Function name: InitPowerOn
Instruction: shr rdx, 8Ch
lea rcx, [rbp+47h+var_00]
xor rdx, rdx
mov dword ptr [rsp+8F0h+var_00], ebx
mov rdx, 80000000h
call BfWInitPhysicalPagesInRange
mov rdx, [rbp+47h+var_00]
mov esi, eax



Rootkit Development



1. User Mode - Kernel Mode Communication

ntddk.h

Toolkit

Communication

2. Direct Kernel Object Manipulation

ntddk.h

Hide Processes

DKOM

3. Keyboard and Mouse Filter

ntddk.h

Keylogger

Keyboard Filter

4. Windows Filtering Platform

fwpmk.h, fwpsk.h, fwpmu.h

Network Control

WFP

5. WinSock Kernel

wsk.h

Network Requests

WSK

6. File System Minifilter Driver

fltKernel.h

Hide Folders

Minifilter

DEMO





COMMUNICATION

User Mode Kernel Mode

console_application.exe

```
// -----  
#define IOCTL_COMM_0 CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800, METHOD_BUFFERED, FILE_ANY_ACCESS)  
  
// -----  
hDevice = CreateFile(L"\\\\.\\MyKernelDriver", GENERIC_READ | GENERIC_WRITE, ...  
  
// -----  
BOOL success = DeviceIoControl(  
    hDevice,  
    * IOCTL_COMM_0,  
    * inBuffer, sizeof(inBuffer),  
    * outBuffer, sizeof(outBuffer),  
    &bytesReturned, NULL  
);
```

IRP

```
typedef struct _IRP {}
```

Handle IRP

```
IRP_MJ_DEVICE_CONTROL
```

Symbolic Link

"\\DosDevices\\MyKernelDriver"

Device Object

"\\Device\\MyKernelDriver"

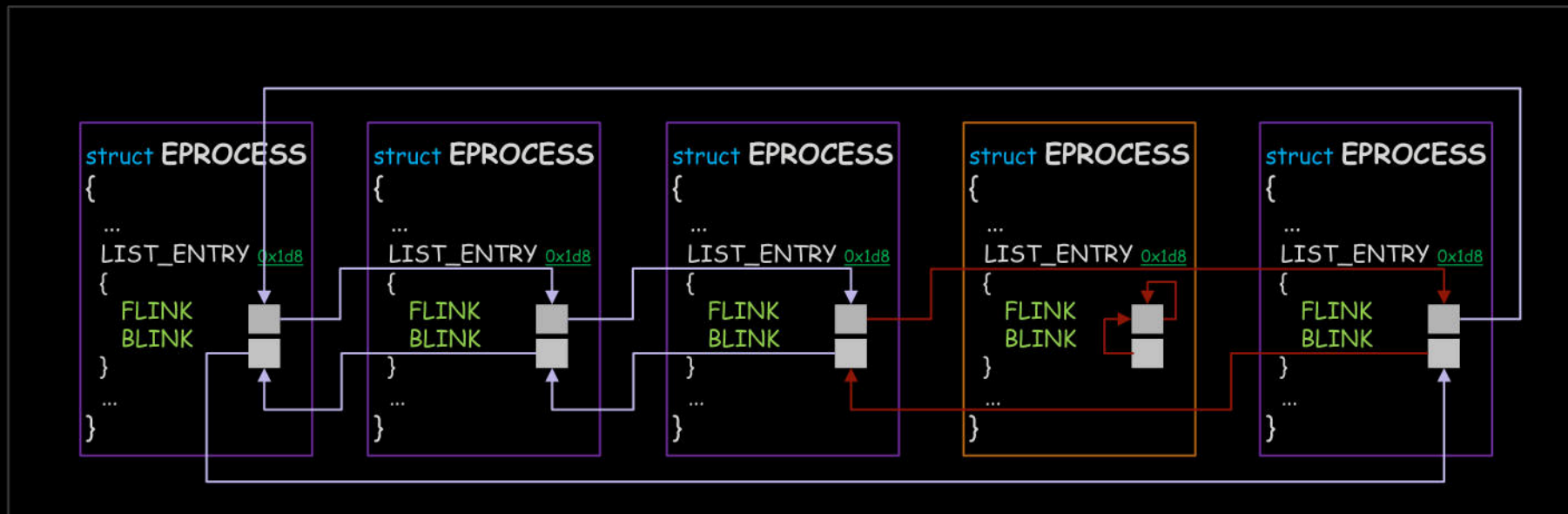
kernel_mode_driver.sys

```
// -----  
#define IOCTL_COMM_0 CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800, METHOD_BUFFERED, FILE_ANY_ACCESS)  
  
// -----  
status = IoCreateDevice(...);  
status = IoCreateSymbolicLink(...);  
  
// -----  
pDriverObject->MajorFunction[...] = ...;  
pDriverObject->MajorFunction[IRP_MJ_DEVICE_CONTROL] = DriverHandleIOCTLs;  
  
// -----  
NTSTATUS  
DriverHandleIOCTLs(  
    _In_ PDEVICE_OBJECT pDeviceObject,  
    _In_ PIRP pIrp  
)  
{  
    PIO_STACK_LOCATION stack = IoGetCurrentIrpStackLocation(pIrp);  
    ULONG controlCode = stack->Parameters.DeviceIoControl.IoControlCode;  
    switch (controlCode)  
    {  
        case IOCTL_COMM_0:  
            ...  
            break;  
        case IOCTL_COMM_1:  
            ...  
            break;  
    }  
    pIrp->IoStatus.Status = STATUS_SUCCESS;  
    IoCompleteRequest(pIrp, IO_NO_INCREMENT);  
    return STATUS_SUCCESS;  
}
```




HIDE PROCESSES

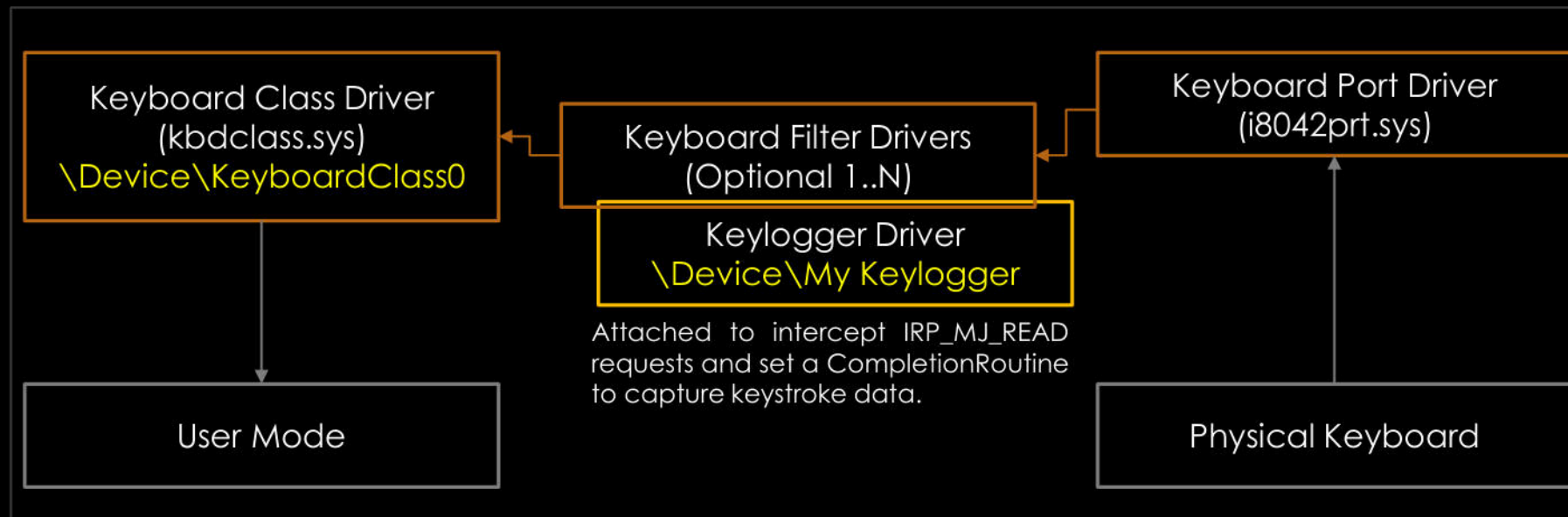
“Windows maintains a doubly linked list of active processes in (LIST_ENTRY) EPROCESS->ActiveProcessLinks. Unlink a process from the chain, and it disappears from user-mode enumeration.”





KEYLOGGER

“Keystroke interception in kernel mode: The Windows keyboard driver stack routes all keystrokes through a device object called `\Device\KeyboardClass0`. By attaching a driver to this device and registering a `CompletionRoutine` (a callback executed after an IRP has been processed by lower drivers, allowing access to data before it reaches the next stage), we can capture raw keystroke data before it propagates to user-mode applications like text editors or browsers.”

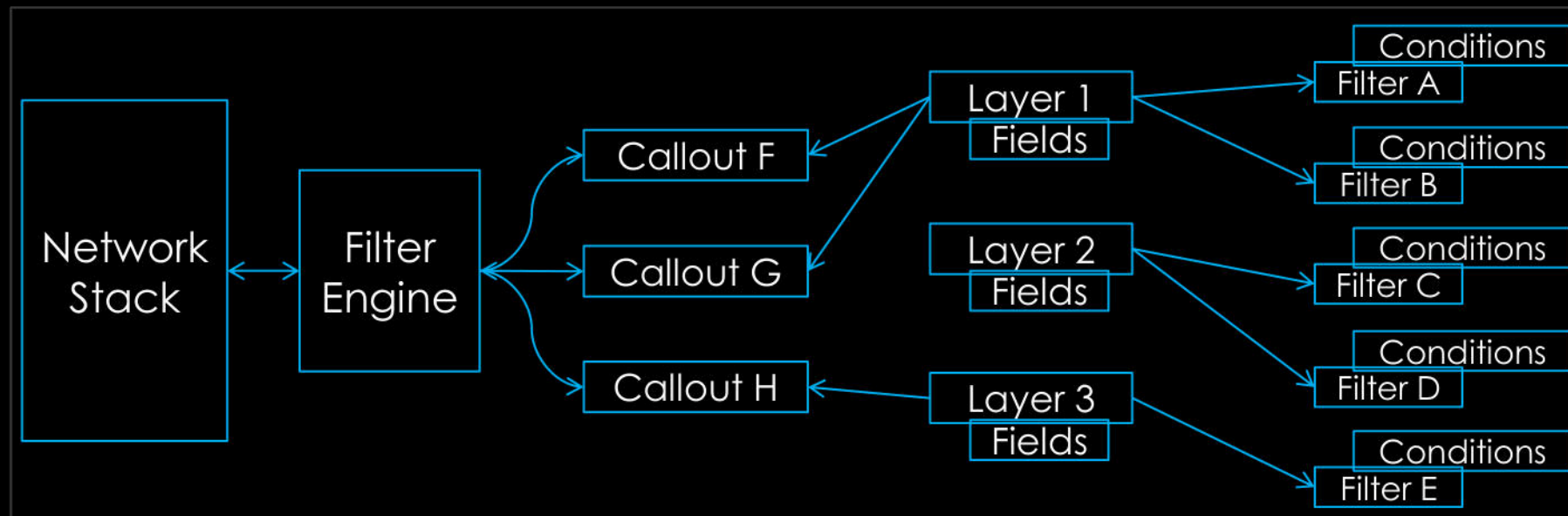




IV

NETWORK CONTROL

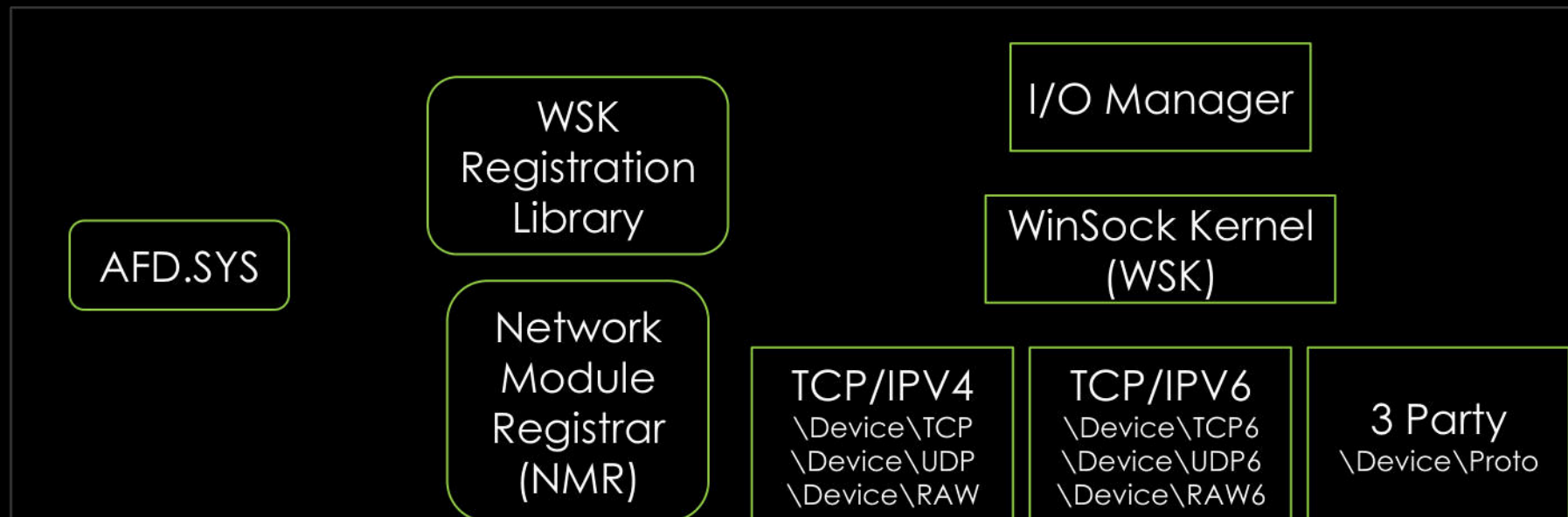
“Windows Filtering Platform (WFP) allows real-time inspection and control of network connections. By attaching filters (static rules applied at specific layers of the network stack to identify traffic based on attributes like IPs or ports) and callouts (custom drivers that execute dynamic logic on flagged traffic), it's possible to classify traffic based on metadata such as the remote IP address and the associated process. Traffic that matches specific rules can be blocked, logged, or modified, enabling comprehensive network security policies.”





NETWORK REQUESTS

“WinSock Kernel (WSK) allows kernel-mode programs to perform complex network operations, such as establishing connections, binding sockets, and transferring data. With support for asynchronous communication using IRPs, WSK enables efficient and controlled interaction with network protocols, ensuring low-latency communication and making it a robust solution for implementing kernel-level networking features.”

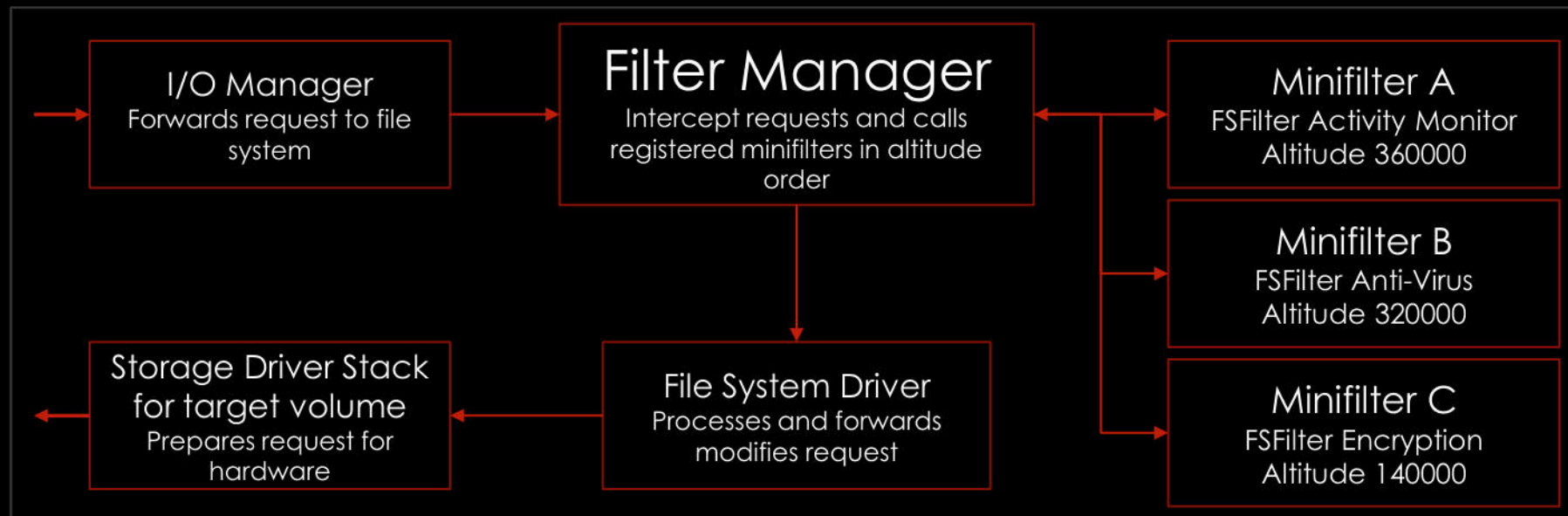


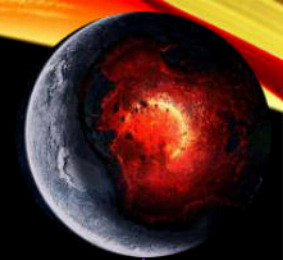


VI

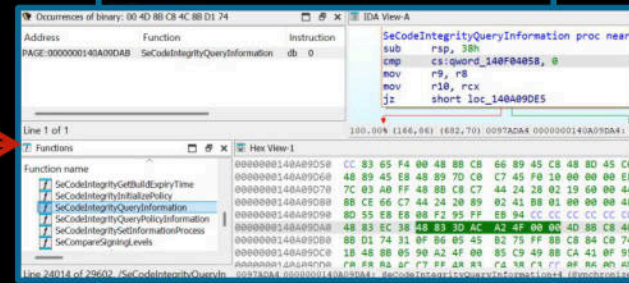
HIDE FOLDERS

“MiniFilters attach to the file system stack to filter I/O operations. Using a PreOperation callback (triggered before the file system processes a request), access to files or directories can be explicitly denied by returning STATUS_ACCESS_DENIED or FLT_PREOP_COMPLETE. In the PostOperation callback (triggered after the request finishes), the DirectoryBuffer - which temporarily holds the directory listing - can be modified to remove specific entries, effectively making files and folders invisible to user-mode applications like File Explorer.”





CmGetSystemDriverList();
SeCodeIntegrityQueryInformation();



Occurrences of binary: 24 01 00 00 EB

Address Function Instruction

```

0x00000000:00000009 OslPeMapKernelSetupPhase1 mov [rdi+124h], r13d
0x00000000:0000000E6B FpCallOpenWellKnownRegist... jr loc_18002D810
0x00000000:0000001800C47E /IsntTimePageTableEntry? lea rax, [rax+124h]
0x00000000:0000001801E5C7 FpExitLbGetFsmatcheP... lea rax, [rax+124h]

```

Line 1 of 4

Functions

Function name Segment S

```

0x00000000:00000000 OslPeMapRuntimeRange .text 0
0x00000000:00000000 OslPeKernelSetupPhase1 .text 0
0x00000000:00000000 OslPeKernelSetupPhase0 .text 0
0x00000000:00000000 OslPeExitFpExitFreeTimePage .text 0
0x00000000:00000000 OslPeExitFpDescriptorCompare .text 0
0x00000000:00000000 OslPeExitFpExitInhibit .text 0

```

Hex View

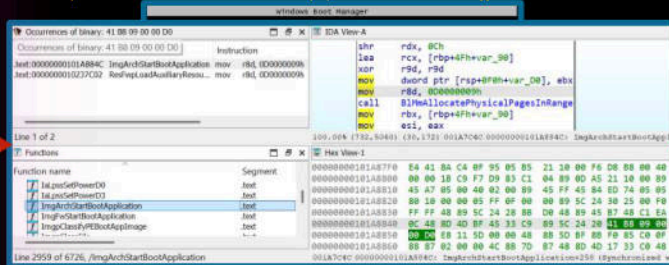
```

00000000:00000000 27 32 0E 0E 02 88 C4 08 83 C4 70 41 5E 5F 5E 5F 5E
00000000:00000009 50 43 CC CC 48 89 C4 24 08 55 53 56 57 41 54 4A
00000000:0000000A 51 46 41 57 47 80 6C 24 E1 48 41 8E 98 08 00
00000000:0000000B 00 45 33 ED 48 80 55 77 48 8B 49 48 69 8F 4
00000000:0000000C 89 60 6F 5C 5D C4 08 55 52 81 4C 48 69 7F 8
00000000:0000000D C8 E8 1A 00 93 00 E4 89 AF 24 08 01 80 98 6
00000000:0000000E 1F 00 45 8A 05 5C A4 79 A7 24 81 8B 05 48 27
00000000:0000000F 4C 80 4D 8F 48 85 25 27 32 0E 4C 80 4D 8F

```

Line 3037 of 7319 (OslPeKernelSetupPhase1)

Windows OS Loader
%SystemRoot%\system32\
winload.efi



Bootkit
UEFI Application
bootmgfw.efi
DXE Runtime Driver
runtime.efi

Rootkit
Kernel-Mode Driver
kmdf.sys

Filter Driver
wfp_wsk_minifilter.sys

Console App
loctl_irp.exe



```
ExitBootServices();
gBS->LoadImage();
```

1. Introduction
2. Overview
3. Boot Manager
4. EFI System Table
5. GUID Partition Table (GPT) Disk Layout
6. Block Translation Table (BTT) Layout
7. Services – Boot Services
8. Services – Runtime Services
9. Protocols - EFI Loaded Image
-
24. Network Protocols – SNP, PXE, BIS and HTTP Boot
25. Network Protocols - Managed Network
26. Network Protocols – Bluetooth
27. Network Protocols – VLAN, EAP, Wi-Fi and Supplicant
28. Network Protocols – TCP, IP, IPsec, FTP, TLS and Configurations
29. Network Protocols – ARP, DHCP, DNS, HTTP and REST
30. Network Protocols – UDP and MTFTP

DEMO



FROM UEFI TO KERNEL





THANK YOU 🐾

UEFI Bootkit

- github.com/TheMalwareGuardian/Abyss

Kernel-Mode Rootkit

- github.com/TheMalwareGuardian/Benthic

Every resource you need to develop Bootkits/Rootkits

- github.com/TheMalwareGuardian/Awesome-Bootkits-Rootkits-Development

Automate Bootkits/Rootkits Development Environment

- github.com/TheMalwareGuardian/Bootkits-Rootkits-Development-Environment

Contact:

- <https://www.linkedin.com/in/vazquez-vazquez-alejandro>
- <https://www.linkedin.com/in/mariasanjose>

💔 In loving memory of Shira
01-01-2013 / 04-07-2025